Arquivos em Lote para a plataforma WINDOWS

Introdução

Arquivos em lote (BATCH) são arquivos de computador utilizados com o intuito de automatizar tarefas, é um modo de processamento no qual os dados de entrada são coletados em grupos (lotes), e periodicamente processados em seqüência por uma ou mais tarefas. O processamento em lote não permite a interação do usuário com o sistema durante a execução dessas tarefas, sendo que os mesmos normalmente são executados em horários agendados.

Quando comparado ao processamento interativo (ONLINE), o processamento em lote costuma se mostrar mais eficiente nos casos de operações rotineiras de alto volume, como o processamento de grandes sistemas da folha de pagamento ou financeiro.

Criação de um arquivo em lote

A criação de um arquivo em lote pode ser feita através de qualquer editor de textos em modo não documento como, por exemplo, o bloco de notas do Windows. Também é de fundamental importância gravar o arquivo com a extensão **.bat**.

Principais Comandos

ECHO

Controla a exibição de mensagens na janela. Os exemplos abaixo mostram as várias opções para utilização do comando:

Para exibir a mensagem "Olá pessoal":

```
ECHO Olá pessoal
```

Desativar as informações exibidas pelo sistema operacional:

```
ECHO OFF
```

Ativar as informações exibidas pelo sistema operacional:

```
ECHO ON
```

Pular uma linha:

```
ECHO.
```

SET

Cria ou modifica uma variável. Para se obter o valor de uma variável é necessário colocá-la entre %. Por exemplo:

```
SET nome=Maria
ECHO Bom dia, %nome%
```

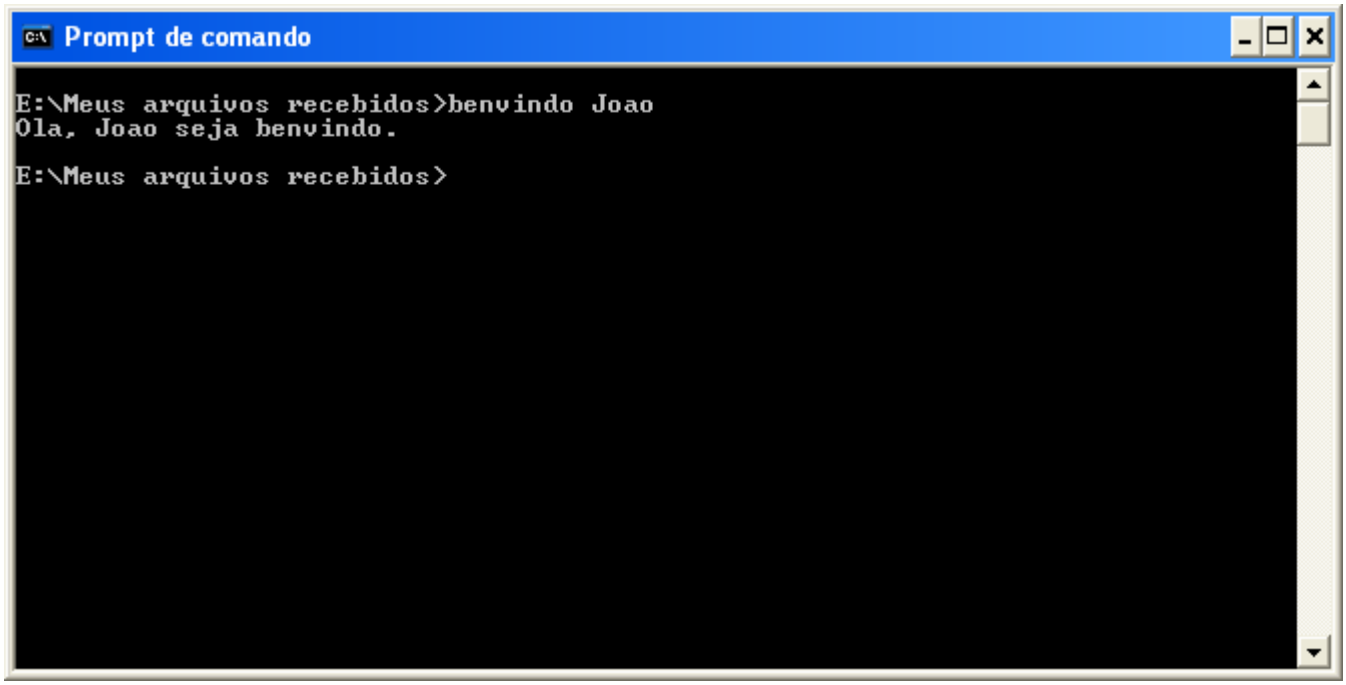
Outra possibilidade do comando SET consiste em permitir uma interação com o usuário, neste caso o parâmetro /P deverá ser utilizado, conforme mostrado no exemplo abaixo:

```
@ECHO OFF
SET /P entrada=Escolha um numero (1-3):
IF %entrada% == 3 GOTO Tres
IF %entrada% == 2 GOTO Dois
IF %entrada% == 1 GOTO Um
GOTO Fim
:Tres
  ECHO Voce pressionou tres.
  GOTO Fim
:Dois
  ECHO Voce pressionou dois.
  GOTO Fim
:Um
  ECHO Voce pressionou um.
:Fim
```

%1, %2, ..., %n
É utilizado para que o arquivo em lotes receba um parâmetro através da linha de comando, por exemplo:

```
@ECHO OFF
ECHO Ola, %1 seja benvindo.
```

Neste exemplo o usuário irá digitar o nome do arquivo em lote e o seu nome e, por exemplo, benvindo João e obterá o seguinte resultado:



:etiqueta
Marcar um local no arquivo em lote, permitindo direcionar (GOTO) a execução a partir do local que foi nomeado.

CLS
Limpar todo o conteúdo exibido na janela.

IF
Comando condicional que permite direcionar o fluxo de execução. No exemplo abaixo o valor da variável período define qual a mensagem que será exibida:

```
@ECHO OFF
SET nome=Maria
SET periodo=Tarde
IF %periodo%==Manha (
  ECHO Bom dia, %nome%!
) ELSE (
  ECHO Boa tarde, %nome%!)
```

GOTO
Direciona a execução para um ponto do arquivo de lote. Por exemplo, GOTO :etiqueta direciona o processamento do programa para :etiqueta.

FOR
Permite realizar a repetição de comandos. A sintaxe do comando é a seguinte:

FOR /L %%variável IN (início, incremento, fim) DO comando

Onde:
/L - opção do comando FOR que realiza os incrementos;
%%variável - nome da variável que irá receber os incrementos;
início - valor de início da variável;
incremento - valor do incremento (passo);
fim - condição de término;
comando - comando que deverá ser repetido.

Abaixo é mostrado um exemplo de utilização do comando:

```
@ECHO OFF
FOR /L %%x IN (1,1,5) DO ECHO Contando ... %%x
```

PAUSE
Pausar o processamento até que alguma tecla seja pressionada.

REM
É comando utilizado para inserir comentários dentro dos arquivos de comando em lote. Por exemplo:

```
REM Isto é um comentário
```

PROMPT
Permite formatar e customizar o sinal de prompt do sistema operacional, sendo que o padrão é pg. As seguintes opções podem ser utilizadas para determinar o formato do prompt:

$p - caminho atual	$g - separador >	$l - separador <	$b - separador |	$q - separador =
$$ - caractere $	$t - hora	$d - data	$h - retrocesso	$n – unidade atual
$v - versão do sistema operacional		$e – símbolo corresponde ao caractere ASCII 27 (ESC)		

DATE
Exibir/ajustar a data do relógio interno do computador.

TIME
Permite exibir/ajustar o horário do relógio interno do computador.

VER
Exibir a versão do sistema sistema operacional em uso.

VOL
Mostrar informações sobre o volume da unidade que foi especificada. Por exemplo: VOL C:

MEM
Exibir informações sobre a utilização da memória RAM, apresenta os seguintes parâmetros opcionais:
/p - exibe a utilização da memória por programas;
/d - exibe a utilização pelos programas e controladores;
/c - realiza a classificação com base na quantidade de memória utilizada.

COPY
É utilizado para realizar a cópia de arquivos. Por exemplo, o comando COPY teste.txt c:\ realiza a cópia do arquivo teste.txt do caminho atual para a pasta c:\

DEL
Permite apagar arquivos. Por exemplo, o comando DEL *.tmp irá apagar todos os arquivos com a extensão .tmp do diretório padrão.

DELTREE
Apaga todos os arquivos e pastas a partir da raiz especificada.

DIR
Exibe uma relação com o conteúdo do diretório especificado.

ATTRIB
Exibir os atributos dos arquivos ou pastas indicados, permitindo também realizar a alteração dos atributos através dos operadores de adição ou subtração. Por exemplo, o comando ATTRIB + r *.exe, faz com que todos os arquivos com a extensão .exe que estão no diretório padrão fiquem marcados apenas como "somente leitura". Os atributos que podem ser utilizados são:
H - oculto R - somente leitura S - sistema A - alterado

RENAME
Alterar o nome de um determinado arquivo. Por exemplo, o comando REN arq1.txt arq2.txt, altera o nome do arquivo arq1.txt para arq2.txt.

TYPE
Exibir o conteúdo do arquivo especificado no parâmetro. Por exemplo, TYPE arq1.txt.

DEFINED
A condição DEFINED verifica se uma variável está declarada. Por exemplo:

```
@ECHO OFF
IF DEFINED nome (
  ECHO Seu nome e %nome%.
) ELSE (
  ECHO Nao sei o seu nome. )
```

EXIST
A condição EXIST verifica a existência de um arquivo. Por exemplo:

```
@ECHO OFF
IF EXIST %1 (
  ECHO O arquivo existe
) ELSE (
  ECHO O arquivo NAO existe )
```

AT

Possibilita agendar um comando, um arquivo em lotes ou um programa para ser executado em data e hora específicas. Também é possível usar este comando para exibir as tarefas que estão agendadas. O comando AT apresenta as seguintes sintaxes:

at \\nome_do_computador hora /interactive | /every:data,... /next:data,... comando
at \\nome_do_computador identificação /delete | /delete/yes

Onde:

- **\\nome_do_computador:** especificar um computador remoto, quando omitido, as tarefas são agendadas para serem executadas no computador local;
- **hora:** especificar a hora em que a tarefa deve ser executada;
- **/interactive:** permitir que a tarefa interaja com a área de trabalho do usuário que está logado no momento em que a tarefa é executada;
- **/every:data,...:** agendar a tarefa para ser executada nos dias especificados da semana ou do mês. Por exemplo, em todas as sextas-feiras ou no oitavo dia de cada mês. Deve-se especificar a data como um ou mais dias da semana (utilizar, para isso, as seguintes abreviações: Seg, Ter, Qua, Qui, Sex, Sab e Dom) ou um ou mais dias do mês (usar números de 1 a 31). As várias entradas de data devem estar separadas com vírgulas. Caso o parâmetro seja omitido, a tarefa é agendada para ser executada no dia atual;
- **/next:data,...:** agendar a tarefa a ser executada na próxima ocorrência do dia (por exemplo, na próxima segunda-feira). É necesspario especificar data como um ou mais dias da semana (usar as seguintes abreviações: Seg, Ter, Qua, Qui, Sex, Sab e Dom) ou um ou mais dias do mês (utilizar os números de 1 a 31). As várias entradas de data devem estar separadas com vírgulas. Caso o parâmetro seja omitido, a tarefa é agendada para ser executada no dia atual;
- **comando:** especificar um comando, um programa (arquivos .exe ou .com) ou um programa em lotes (arquivo .bat ou .cmd) que deverá ser executado. Quando o comando não é um arquivo executável (.exe), é necessário colocar CMD /C antes do comando. Por exemplo, CMD /C "COPY C:*.* C:\temp";
- **identificação:** especificar o número de identificação que foi atribuído a uma tarefa agendada;
- **/delete:** cancelar uma tarefa agendada. Caso o parâmetro identificação esteja omitido, todas as tarefas que estão agendadas no computador são canceladas;
- **/yes:** forçar uma resposta positiva para todas as consultas do sistema quando for cancelar tarefas agendadas. Caso omita o parâmetro, o usuário será solicitado a confirmar o cancelamento de cada uma das tarefas agendadas.

Exemplo:

```
AT 16:27 CMD /C "DIR c:\*.* > c:\teste.txt"
```

Interação entre as aplicações e o sistema operacional

É possível realizar a troca de informações entre um determinado programa e o sistema operacional, no nosso caso o Windows. Considerando o programa DIR, é possível que ele receba informações (parâmetros) do sistema operacional (*.doc). Por exemplo: DIR *.doc – comando este que irá realizar a exibição dos arquivos que tenham a extensão .doc e que estejam no diretório corrente.

No sentido inverso também é possível a troca de informações, neste caso, o programa consegue enviar para o sistema operacional o resultado da sua execução. Isto é possível através da variável de ambiente **errorlevel**, a qual poderá ser tratada posteriormente pelo sistema operacional.

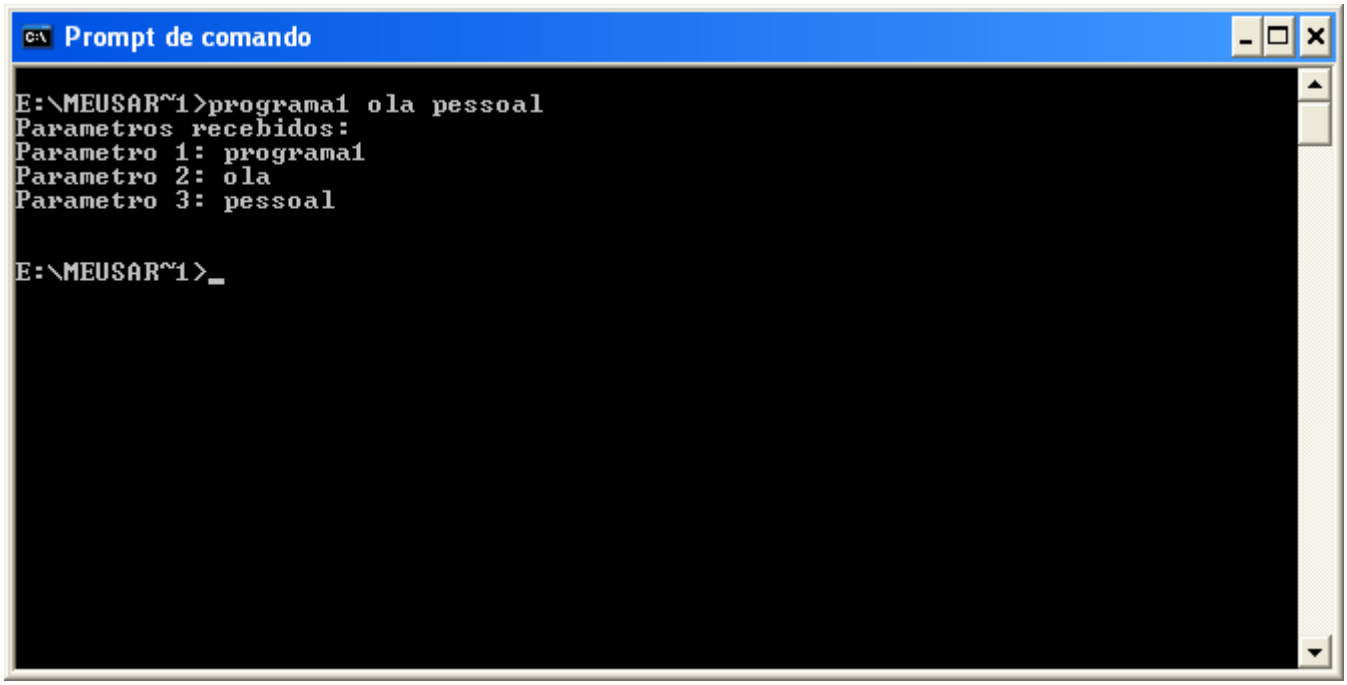
Exemplos em Linguagem "C"

A linguagem "C" através da função main pode receber parâmetros através da linha de comando do sistema operacional. Ao término do processamento também pode retornar um valor inteiro o qual poderá ser obtido pelo sistema operacional através da variável de ambiente **errorlevel**. O exemplo a seguir ilustra esses conceitos:

```
// programa1.cpp
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{ printf("Parametros recebidos:\n");
  for (int i = 0; i < argc; i++)
  { printf("Parametro %d: %s\n", (i + 1), argv[i]);
  }
  printf("\n");
  return (argc);
}
```

O resultado da execução do programa a partir do **Prompt de comando** é o seguinte:



Após a compilação e geração do arquivo executável, tratado neste exemplo como programa1, é possível escrever um arquivo em lote que envie e recupere as informações do programa, por exemplo:

```
@ECHO OFF
CLS
programa1 %1 %2 %3
IF ERRORLEVEL 3 GOTO par3
IF ERRORLEVEL 2 GOTO par2
IF ERRORLEVEL 1 GOTO par1

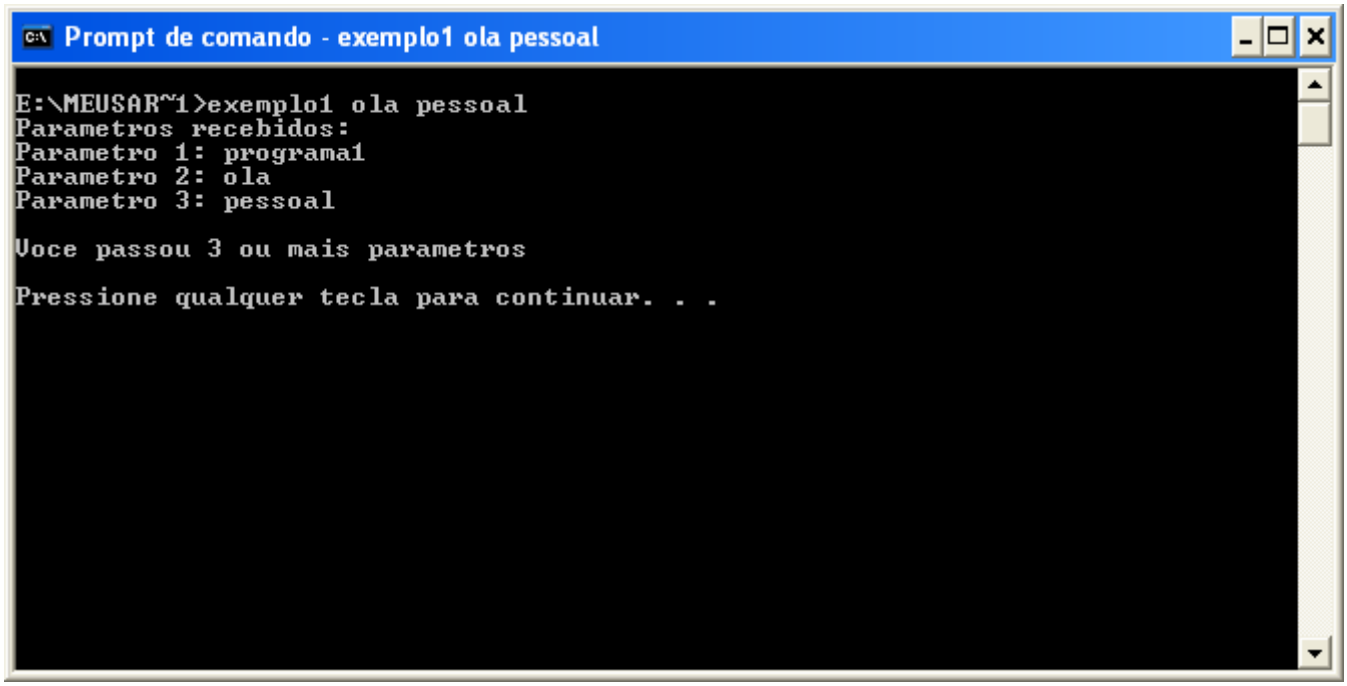
:par3
  ECHO Voce passou 3 ou mais parametros
  GOTO fim

:par2
  ECHO Voce passou 2 parametros
  GOTO fim
```

```
:par1
  ECHO Voce passou 1 parametro
  GOTO fim

:fim
  ECHO.
  PAUSE
  ECHO ON
```

O código acima deverá ser gravado com o nome exemplo1.bat sendo, que a sua execução irá produzir o seguinte resultado:



Neste outro exemplo será criado um programa que deverá receber do sistema operacional um nome de arquivo e, em seguida será exibido o conteúdo do mesmo com a exibição dos respectivos números de linhas.

```
// exibir.cpp
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>

#define TAMANHO_LINHA 1000

int main(int argc, char *argv[])
{ int erro = 0;
  char linha[TAMANHO_LINHA];
  int i = 1;
  FILE *arquivo;
  if (argc == 2)
  { if ((arquivo = fopen(argv[1], "r"))==0)
    { printf("\nERRO - Nao foi possivel abrir o arquivo %s.", argv[1]);
      erro = 2;
    }
    else
    { while (fgets(linha, TAMANHO_LINHA, arquivo) != NULL)
      { printf ("%3d: %s", i, linha);
        i++;
      }
      fclose(arquivo);
```

```
    }
  }
  else
  { printf("\nATENCAO - Esse programa deve ser usado da seguinte forma:\nexibir
    nome_do_arquivo.extensao\n");
    erro = 1;
  }
  system ("PAUSE");
  return (erro);
}
```

A execução deste programa irá produzir o seguinte resultado:

